CodeMentor AI

Eine hybride Code-Mentoring-Engine, die Code nicht repariert, sondern prüft und sokratische Fragen stellt.

PYTHON 3.12 | 3.700+ TESTS | MYPY --STRICT | KEIN DOCKER NÖTIG | MIT-LIZENZ

Auf einen Blick

Merkmal	Beschreibung
Sprachen & Kontext	C, C++, C#, Python, HTML, CSS + Unity- und Unreal-Engine-Kontext
Statische Regeln	41 deterministische Regeln (Syntax, Speicherlecks, Big-O, Sicherheit)
Vorvalidierung	tree-sitter (Tier 0) + gcc / clang / Roslyn (Tier 1 Syntaxprüfung)
LLM-Unterstützung	Claude Code CLI, Codex, Anthropic API, Gemini, Ollama (lokal)
Schnittstellen	Windows Desktop (.exe), React Web UI, CLI, REST API + SSE
Evaluierung	166 kuratierte Beispiele (dev 96 / test 70), 0% Lösungs-Leak, 100% Injection-Schutz

Warum CodeMentor AI?

Fragt ein Lernender ein Standard-LLM "was ist an meinem Code falsch?", liefert dieses meist direkt die fertige Lösung. Der Lernende kopiert den Code ohne Erkenntnisgewinn. CodeMentor AI kehrt dieses Prinzip um:

- **Fragen statt Lösungen:** LLM-Antworten werden niemals ungefiltert ausgeliefert. Ein Leakage Guard verwirft Fragen, die fertigen Code verraten (0% Leakage).
- **Deterministische Basis:** Bei Syntaxfehlern bricht die Pipeline sofort ab (~1 ms), ohne LLM-Kosten. Bei LLM-Ausfall erfolgt automatisches Graceful Degradation auf statische Berichte.
- **Progressive Hinweise (1 → 3):** Stufe 1 (Konzeptioneller Hinweis) → Stufe 2 (Zeilenbezug) → Stufe 3 (Richtungswegweiser). Die Lösung wird niemals vorgegeben.
- **Sicherheit & Isolation:** Benutzercode wird strikt als ungesichert eingestuft und ohne Ausführung in isolierten Prozessen validiert.

Was der Lernende sieht

1. Syntaxfehler (Früher Abbruch, 0 LLM-Tokens, ~1 ms)

```
int summe = 0 // Fehlendes Semikolon
```

Mentorenfrage: "Ein Ausdruck in Zeile 4 scheint nicht mit ';' zu enden. Welches Zeichen erwartet der Compiler an dieser Stelle?"

2. Speicherleck & Statische Regel (C-MEM-001)

```
int *tmp = malloc(n * sizeof(int)); // Kein free()
```

Sokratische Frage (Stufe 1): "Was passiert mit dem während der Ausführung angeforderten Heap-Speicher nach dem Verlassen der Funktion, und wer kann ihn freigeben?"

Engineering & Evaluierungsergebnisse

Metrik	Claude Opus (two_pass)	Nur Statisch (Kein LLM)
Audit Präzision / Recall / F1 (dev 96)	**79 / 97 / 87**	100 / 57 / 73
Früher Abbruch (Precision/Recall)	100% / 100%	100% / 100%
Big-O Komplexitätsgenauigkeit	97% / 97%	97% / 97%

Lösungsleckage (Leakage)	**0%**	—
Prompt-Injection-Resistenz	**100%**	—